

1. Vue d'ensemble du système de rôles

1.1 Rôles détectés

Source: backend/app/models.py, lignes 41-47 — enum UserRole(str, Enum)

Valeur technique	Nom fonctionnel	Description
member	Membre générique	Rôle non documenté, jamais vérifié dans les endpoints
student	Étudiant	Rôle par défaut à l'inscription
teacher	Enseignant	Enseignant de l'école (possède school_id)
admin	Admin legacy	Marqué "for migration" — peut toujours être utilisé
admin_school	Admin d'école	Admin scoping à une école spécifique
super_admin	Super Admin	Accès global à toute la plateforme

1.2 Double indicateur de super-admin

Source: backend/app/models.py, ligne 183

Le modèle User possède deux indicateurs indépendants :

- role = UserRole.SUPER_ADMIN (enum)
- is_super_admin = Column(Boolean, default=False) (booléen)

Incohérence: require_super_admin dans learner.py (ligne 28) vérifie is_super_admin OR role == "SUPER_ADMIN", tandis que tous les autres routers ne vérifient que le role string. Un utilisateur avec role="student" et is_super_admin=True passerait le check learner.py mais échouerait dans admin.py.

1.3 Rôle MEMBER

Source: backend/app/models.py, ligne 43

Le rôle MEMBER existe dans l'enum mais n'est jamais vérifié dans aucun endpoint ni dépendance. Il est listé comme valide dans change_user_role (admin.py, ligne 607) mais n'a aucune permission définie.

2. Fiche détaillée par rôle

2.1 SUPER_ADMIN

Fonctionnalités autorisées:

Fonctionnalité	Endpoint	Fichier:Ligne
CRUD complet écoles	POST/GET/PUT/DELETE /schools	admin.py:101-113, 116-125, 128-137, 1755-1775
CRUD complet utilisateurs (global)	GET/POST/PUT/DELETE /users*	admin.py:286-321, 522-545
Changement de rôle utilisateur	PUT /users/{id}/change-role	admin.py:607-633
Réinitialisation mot de passe	POST /users/{id}/reset-password	admin.py:273-283
Suppression utilisateur	DELETE /users/{id}	admin.py:1445-1451
Gestion finance globale	GET/POST /wallets/*, /transactions	admin.py:1340-1350, 1353-1370
Stats globales	GET /stats/global	admin.py:1386-1396
Settings plateforme	PUT /settings, /settings/apply	admin.py:943-967, 1050-1059
Maintenance mode	Via settings	admin.py

Fonctionnalité	Endpoint	Fichier:Ligne
Audit logs	GET /audit-logs	admin.py:1840-1880
Distribution cours B2B	POST/DELETE /schools/{id}/grant/revoke-course	admin_courses.py:860-909
AI Factory complet	POST /api/admin/ai-factory/*	ai_factory.py:1-337
Preview cours learner	GET /learner/courses/{id}/preview	learner.py:36-100
Voir tous les cours (draft inclus)	Via syllabus endpoint	learner.py:240-274

2.2 ADMIN_SCHOOL

Fonctionnalités autorisées:

Fonctionnalité	Endpoint	Fichier:Ligne
Gestion école assignée (sans cross-school)	GET/PUT /schools/{id}	admin.py (⚠ pas de vérification cross-school)
Gestion utilisateurs de son école	GET/POST /users	admin.py:286-321
Gestion classes de son école	GET/POST /classes	admin.py:449-462, 465-495
Enrollments de son école	GET/POST /enrollments	admin.py:498-510, 512-519

Fonctionnalité	Endpoint	Fichier:Ligne
Dashboard école	GET /dashboard/stats, /dashboard	admin.py:1160-1215
Gestion cours (school-scoped)	Tous /admin/courses/*	admin_courses.py (via _can_access_course)
AI Factory (publie dans son école)	POST /api/admin/ai-factory/*	ai_factory.py

Restrictions:

- Ne peut PAS gérer les settings plateforme (⚠️ mais le code ne le vérifie pas — voir failles)
- Ne peut PAS supprimer d'écoles
- Ne peut PAS voir les stats globales

2.3 TEACHER

Fonctionnalités autorisées:

Fonctionnalité	Endpoint	Fichier:Ligne
Cours propres (CRUD)	GET/POST/PUT /courses/*	courses.py
Modules et leçons de ses cours	POST /courses/{id}/modules, /modules/{id}/lessons	courses.py:128-148
Classroom management	GET/POST /lms/classes/*	lms.py:575-582
Assignments dans ses classes	POST /lms/assignments	lms.py:30-37
Noter les soumissions	PUT /lms/submissions/{id}/grade	lms.py:230-241

Fonctionnalité	Endpoint	Fichier:Ligne
AI Tutor (ask, explain, quiz)	POST /ai/ask, /ai/explain, /ai/quiz	ai.py:31-103
AI Studio (generate)	POST /ai/generate	ai.py:255-325
Ingest PDF/text (son école)	POST /ai/ingest/*	ai.py:128-213
Wallet personnel	Via /lms/wallet	lms.py

2.4 STUDENT

Fonctionnalités autorisées:

Fonctionnalité	Endpoint	Fichier:Ligne
Catalogue cours public	GET /catalog/courses	catalog.py:15-68
Détail cours public	GET /catalog/courses/{slug}	catalog.py:69-120
S'inscrire à un cours	POST /learner/courses/{id}/enroll	learner.py:279-307
Voir mes cours	GET /learner/my-courses	learner.py:310-333
Voir leçons	GET /learner/lessons/{id}	learner.py:338-392
Progression	POST /learner/lessons/{id}/progress	learner.py:395-489
Quiz	POST /learner/quizzes/{id}/start, /submit	learner.py:492-600
Certificats	GET /learner/courses/{id}/certificate	learner.py:716-758

Fonctionnalité	Endpoint	Fichier:Ligne
Notes	GET/POST /learner/lessons/{id}/notes	learner.py:763-783
AI Tutor	POST /ai/ask, /ai/explain	ai.py:31-63
Soumettre devoirs	POST /lms/assignments/{id}/submit	lms.py:127-160

Restrictions:

- Ne peut PAS créer/modifier/supprimer des cours
- Ne peut PAS gérer les utilisateurs
- Ne peut PAS ingester du contenu IA

2.5 ADMIN (legacy)

Identique à ADMIN_SCHOOL pour les permissions. Le rôle est marqué "for migration" mais toujours fonctionnel dans les checks.

3. Matrice Rôles × Fonctionnalités

Fonctionnalité	SUPER_ADMIN	ADMIN_SCHOOL	TEACHER	STUDENT
Gérer écoles (CRUD)	✓	✗	✗	✗
Supprimer école	✓	✗	✗	✗
Gérer tous les utilisateurs	✓	◆ (école)	✗	✗
Créer un utilisateur	✓	◆ (école)	✗	✗
Changer rôle utilisateur	✓	✗	✗	✗

Fonctionnalité	SUPER_ADMIN	ADMIN_SCHOOL	TEACHER	STUDENT
Reset mot de passe	✓	◆ (école)	✗	✗
Toggle actif/inactif	✓	◆ (école)	✗	✗
Voir audit logs	✓	✗	✗	✗
Settings plateforme	✓	⚠ (non vérifié)	✗	✗
Distribution cours B2B	✓	✗	✗	✗
AI Factory	✓	✓	✗	✗
Stats globales	✓	✗	✗	✗
Dashboard admin	✓	◆ (école)	✗	✗
CRUD cours (propriétaires)	✓	✓	✓	✗
Gérer classes	✓	◆ (école)	◆ (siennes)	✗
Noter devoirs	✓	✓	✓	✗
Ingest contenu IA	✓	✓	✓	✗
AI Tutor / Générer contenu	✓	✓	✓	✓

Fonctionnalité	SUPER_ADMIN	ADMIN_SCHOOL	TEACHER	STUDENT
Catalogue public	✓	✓	✓	✓
S'inscrire à un cours	✓	✓	✓	✓
Progression / Quiz	✓	✓	✓	✓
Certificats	✓	✓	✓	✓
Wallet personnel	✓	✓	✓	✓

4. Règles d'authentification par rôle

4.1 Mécanisme d'authentification

Source: backend/app/auth.py

Aspect	Valeur
Méthode	JWT Bearer Token (OAuth2PasswordBearer)
Algorithme	HS256 (HMAC-SHA256) — config.py:17
Secret JWT	Variable d'environnement JWT_SECRET — auto-généré en dev
Durée token	15 minutes (hardcodée, non configurable) — auth.py:25
Payload JWT	{"sub": user_id, "school_id": int, "exp": datetime}
Hashing mots de passe	bcrypt (work factor par défaut) — security.py:3

Aspect	Valeur
Refresh token	Non implémenté
2FA	Non implémenté
Lockout compte	Non implémenté

4.2 Incohérence critique: expiry config morte

Source: backend/app/core/config.py, ligne 18

```
jwt_expire_minutes: int = 1440 # 24 heures
```

Cette valeur n'est jamais utilisée. La fonction `create_access_token()` (auth.py:25) utilise un fallback hardcodé de 15 minutes. Les appelsants (register line 85, login line 102) ne passent jamais `expires_delta`.

4.3 Vérification `is_active` manquante

Source: backend/app/auth.py, lignes 38-52

La fonction `get_current_user()` vérifie `user is None` mais ne vérifie jamais `user.is_active`. Un utilisateur désactivé peut toujours s'authentifier et accéder à tous les endpoints.

4.4 Pas de différence de sécurité entre rôles

Tous les rôles utilisent le même mécanisme JWT avec la même durée de vie. Il n'y a pas d'authentification renforcée pour les admins.

5. Flux d'inscription et attribution des rôles

5.1 Inscription publique

Source: backend/app/auth.py, lignes 55-86

```
@router.post("/register", response_model=Token)
```

```
def register(user_in: UserCreate, db: Session = Depends(get_db)):
```

```
    role = user_in.role or "student" # LINE 74 — role accepté directement
```

```
    new_user = User(..., role=role) # LINE 81
```

FAILLE CRITIQUE — Élévation de privilèges: Le champ `role` du schéma `UserCreate` (schemas.py:19) accepte n'importe quelle string sans validation. Un utilisateur peut s'inscrire avec `role: "super_admin"` et obtenir tous les privilèges.

5.2 Inscription enseignant

Source: backend/app/routers/admin.py, lignes 1690-1790

Les enseignants s'inscrivent via un formulaire qui crée un TeacherRegistration (table séparée). Un admin doit approuver via PUT /teacher-registrations/{id}/review. Le champ is_approved (models.py:190) contrôle l'accès.

5.3 Attribution par admin

Source: backend/app/routers/admin.py, lignes 286-321, 607-633

Un admin peut :

- Créer un utilisateur avec un rôle (POST /users) — aucune validation du rôle
- Changer le rôle d'un utilisateur (PUT /users/{id}/change-role) — valide contre une whitelist

6. Failles et incohérences détectées

CRITIQUE

#	Faille	Fichier:Ligne	Impact
C1	Élévation de privilèges par auto-inscription — role accepté sans validation dans /register	auth.py:74, schemas.py:19	Tout utilisateur peut devenir super-admin
C2	is_active non vérifié à l'authentification — les utilisateurs désactivés conservent l'accès	auth.py:38-52	Comptes super-admin restent actifs
C3	Mots de passe sans complexité — validate_password_strength() existe (validation.py:82) mais n'est jamais appelé	schemas.py:17, auth.py:55	Mots de passe faibles ou trivial acceptés
C4	Identifiants DB hardcodés — mot de passe PostgreSQL gill4264 dans 18+ scripts	scripts/reset_db.py:3, etc.	Exposition d'identifiants de base de données dans le code source
C5	Admin school peut modifier les settings plateforme — pas de vérification super_admin	admin.py:943, 1050	Un admin d'école peut activer/désactiver le site

#	Faible	Fichier:Ligne	Impact
			maintenanc ou changer

HAUTE

#	Faible	Fichier:Ligne	Impact
H1	Pas de lockout après échecs de connexion — brute force possible	auth.py:89-103	Attaque par force brute
H2	Pas de 2FA — aucun mécanisme d'authentification à deux facteurs	N/A	Compromission de compte plus facile
H3	Pas de refresh token — tokens expirent en 15 min sans renouvellement	auth.py:25	Mauvaise UX, tokens trop courts
H4	Rate limiting en mémoire — ne survit pas au redémarrage, ne fonctionne pas multi-worker	main.py:73-93	Contournable
H5	Cross-school non vérifié sur plusieurs endpoints admin	admin.py:GET /schools/{id}, GET /users/{id}, PUT /toggle-active	Admin d'école A accède aux données de l'école B
H6	POST /lms/classes/{id}/assignments sans vérification de rôle — tout utilisateur authentifié peut créer des devoirs	lms.py:514-550	Un étudiant peut créer des devoirs

#	Faible	Fichier:Ligne	Impact
H7	GET /lms/classes/{id}/students sans scoping — tout utilisateur peut lister les étudiants d'une classe	lms.py:443, 485	Fuite de données cross-école
H8	GET /admin/quizzes/{quiz_id} sans vérification _can_access_lesson	admin_quizzes.py:137-142	Admin d'école A voit les quiz de l'école B
H9	Pas de headers de sécurité — pas de HSTS, CSP, X-Frame-Options	main.py	Vulnérabilités XSS/clickjacking
H10	Docs Swagger toujours exposées — /docs et /openapi.json en production	main.py:131-133	Surface d'attaque exposée

MOYENNE

#	Faible	Fichier:Ligne
M1	4 copies indépendantes de require_admin — risque de divergence	admin.py:58, admin_courses.py:30, adm
M2	JWT expiry config morte — jwt_expire_minutes=1440 jamais utilisé	config.py:18 vs auth.py:25
M3	Rôle MEMBER défini mais jamais vérifié	models.py:43
M4	is_super_admin booléen redondant avec le role enum	models.py:183
M5	Sanitization définie mais jamais utilisée — sanitize_string(), sanitize_html()	validation.py:8,57

#	Faible	Fichier:Ligne
M6	Audit logging défini mais jamais appelé — <code>log_admin_action()</code> , <code>log_security_event()</code>	<code>audit.py</code> :9,54
M7	Password reset envoyé en query parameter — logged dans les accès serveur	<code>admin.py</code> :274
M8	Rôle stocké en <code>localStorage</code> mutable — le frontend peut être contourné	<code>store/authStore.ts</code> :29
M9	Incohérence casse des rôles — certains routers lowercase, d'autres UPPERCASE	<code>lms.py</code> :17-19 vs <code>admin_courses.py</code> :24-2
M10	CORS hardcoded localhost — pas de config production	<code>main.py</code> :104-113
M11	Logger level WARNING pour login — logge l'email et l'objet User complet	<code>auth.py</code> :95,97

7. Recommandations d'amélioration

Priorité CRITIQUE (à corriger immédiatement)

1. Valider le champ `role` à l'inscription — Restreindre à `["student"]` uniquement dans `auth.py`:74. Les rôles élevés ne doivent être attribuables que par un admin via `/users/{id}/change-role`.
2. Vérifier `user.is_active` dans `get_current_user()` — Ajouter `if not user.is_active: raise HTTPException(403, "Account disabled")` dans `auth.py`:50.
3. Appliquer `validate_password_strength()` — Appeler la fonction existante dans `/register` et `/reset-password`.

4. Restreindre les settings plateforme au super_admin — Ajouter une vérification is_super_admin dans les endpoints PUT /settings et POST /settings/apply.
5. Ajouter la vérification cross-school sur GET /schools/{id}, GET /users/{id}, PUT /toggle-active, GET /quizzes/{id}.

Priorité HAUTE (à planifier)

6. Implémenter un lockout de compte — 5 tentatives échouées → blocage 15 min. Utiliser Redis ou la DB.
7. Ajouter la 2FA — Optionnelle pour les students, obligatoire pour les admins.
8. Implémenter un refresh token — Rotation de token, durée d'accès 1h, refresh 7j.
9. Corriger le rate limiting — Utiliser Redis au lieu de la mémoire en processe.
10. Ajouter les headers de sécurité — HSTS, CSP, X-Frame-Options, X-Content-Type-Options.
11. Masquer /docs et /openapi.json en production — docs_url=None si is_production.

Priorité MOYENNE (à améliorer)

12. Centraliser require_admin — Une seule définition dans un module partagé (ex: app/deps.py).
13. Utiliser la config jwt_expire_minutes — Passer expires_delta aux endpoints register/login.
14. Nettoyer les identifiants hardcodés — Utiliser des variables d'environnement pour tous les scripts.
15. Activer l'audit logging — Appeler log_admin_action() et log_security_event() aux bons endroits.
16. Ajouter CORS dynamique — Lire les origines depuis la config/DB au lieu de les hardcoder.
17. Ajouter des checks de rôle côté frontend sur les routes teacher/student — RequireRole au lieu de RequireAuth.

Le rapport complet est ci-dessus. Les 3 failles les plus critiques à corriger en priorité :

1. C1 — auth.py:74 : Le champ role est accepté sans validation à l'inscription. N'importe qui peut s'inscrire en super_admin.
2. C2 — auth.py:50 : is_active n'est jamais vérifié. Un utilisateur désactivé garde accès à tout.

3. C5 — admin.py:943 : Un admin d'école peut modifier les settings plateforme (maintenance mode, clés AI) — devrait être réservé au super_admin.